

Memoria de la Fase 1

Diseño Jerárquico y Conversores de Código

Diseño Digital 2

5 de mayo de 2026

1. Introducción y Objetivos

El objetivo de esta primera fase ha sido la realización de la propuesta de diseño jerárquico y el modelado y depuración de los bloques conversores de código: **BCD a Binario** y **Binario a BCD**.

Estos módulos son piezas fundamentales de la calculadora, ya que el sistema debe procesar operandos de entrada de tres dígitos decimales (comprendidos entre -999 y 999), mientras que las operaciones aritméticas internas —suma, resta y multiplicación— se realizan en formato binario en complemento a dos. La interfaz de usuario, sin embargo, trabaja en BCD, lo que hace imprescindible disponer de conversores fiables en ambas direcciones.

2. Conversor BCD a Binario (`bcd_to_bin`)

2.1. Especificación

Este módulo implementa lógica **combinacional pura**. Recibe un vector BCD de 12 bits organizado en tres nibbles: centenas `bcd[11:8]`, decenas `bcd[7:4]` y unidades `bcd[3:0]`, y produce un vector binario de 10 bits capaz de representar valores entre 0 y 999.

2.2. Implementación

Siguiendo las directrices de la asignatura, se emplea el paquete `ieee.std_logic_unsigned`. Dado que dicho paquete **no soporta la multiplicación directa** de vectores de longitud arbitraria, la operación $\text{bin} = c \times 100 + d \times 10 + u$ se implementa mediante **desplazamientos y sumas**, que son la representación hardware natural de las potencias de dos:

$$c \times 100 = c \times 64 + c \times 32 + c \times 4 = (c \ll 6) + (c \ll 5) + (c \ll 2)$$

$$d \times 10 = d \times 8 + d \times 2 = (d \ll 3) + (d \ll 1)$$

Cada desplazamiento se implementa en VHDL mediante concatenación de ceros, lo que garantiza que no se introduzcan operadores no soportados por la librería.

El fragmento central de la arquitectura es el siguiente:

```

1  -- c * 100 = c*64 + c*32 + c*4
2  c100 <= (c(3 downto 0) & "000000") -- c * 64
3      + (c(4 downto 0) & "00000")  -- c * 32
4      + (c(7 downto 0) & "00");    -- c * 4
5

```

```

6  -- d * 10 = d*8 + d*2
7  d10  <= (d(6 downto 0) & "000")      -- d * 8
8        + (d(8 downto 0) & '0');      -- d * 2
9
10 bin <= c100 + d10 + u;

```

Listing 1: Multiplicación por desplazamientos en bcd_to_bin

3. Conversor Binario a BCD (bin_to_bcd)

3.1. Especificación

La calculadora debe poder mostrar resultados de hasta $999 \times 999 = 998\,001$, por lo que el conversor recibe un número binario de **20 bits** y genera **6 dígitos BCD** (24 bits). Un diseño combinacional sería inviable a esta escala, por lo que se adopta una **arquitectura secuencial iterativa**.

3.2. Algoritmo: Double Dabble

Se implementa el algoritmo *Double Dabble* (desplazamiento y suma de 3), que procesa el número binario bit a bit, del MSB al LSB, en **20 ciclos de reloj**. Cada ciclo consta de dos pasos:

1. **Add-3:** Para cada nibble BCD del acumulador, si su valor es ≥ 5 , se le suma 3. Esto garantiza que tras el desplazamiento siguiente el nibble no desborde el dígito decimal.
2. **Shift izquierda:** Se desplaza un bit a la izquierda el registro combinado [acc_bcd | bin_reg], introduciendo el MSB de bin_reg por la derecha de acc_bcd.

```

1  -- Paso 1: Add-3 a cada nibble si >= 5
2  if acc_bcd(3 downto 0) >= "0101" then n0 := acc_bcd(3 downto 0) + "0011";
3  else n0 := acc_bcd(3 downto 0); end if;
4  -- (idem para n1..n5)
5
6  -- Paso 2: Reconstruir vector ajustado y desplazar
7  bcd_adj := n5 & n4 & n3 & n2 & n1 & n0;
8  acc_bcd <= bcd_adj(22 downto 0) & bin_reg(N_BITS-1);
9  bin_reg <= bin_reg(N_BITS-2 downto 0) & '0';

```

Listing 2: Núcleo del algoritmo Double Dabble en bin_to_bcd

3.3. Máquina de Estados

El control se implementa mediante una FSM de tres estados, tal y como se resume en la Tabla 1.

Estado	Descripción
IDLE	Espera activa de start='1'. Pone done='0' y carga bin_reg.
CALC	Ejecuta un paso del algoritmo por ciclo durante N_BITS ciclos.
FINISH	Activa done='1' durante un ciclo y retorna a IDLE.

Cuadro 1: Estados de la FSM del conversor bin_to_bcd.

La señal `done` se mantiene a '1' mientras el sistema permanece en `IDLE` y se pone a '0' en el mismo ciclo en que `IDLE` detecta un nuevo `start='1'`, evitando condiciones de carrera en el banco de pruebas.

4. Depuración y Bancos de Pruebas

4.1. `tb_bcd_to_bin` — Combinacional

Al tratarse de lógica combinacional, el testbench aplica estímulos directamente mediante asignaciones a `bcd_in` separadas por `wait for 20 ns`. No requiere reloj. Los casos de prueba cubren los límites y valores representativos recogidos en la Tabla 2.

BCD entrada (hex)	Binario esperado (dec)	Propósito
0x000	0	Cero absoluto
0x001	1	Unidad mínima
0x010	10	Solo decenas
0x100	100	Solo centenas
0x123	123	Valor mixto
0x256	256	Potencia de 2
0x999	999	Máximo absoluto

Cuadro 2: Casos de prueba del testbench `tb_bcd_to_bin`.

4.2. `tb_bin_to_bcd` — Secuencial

El testbench secuencial genera un reloj de **50 MHz** (periodo de 20 ns), cumpliendo la especificación no funcional **ESP15**. Para garantizar una detección fiable de `done='1'` sin condiciones de carrera, se emplea un **procedimiento auxiliar** (`run_test`) que alinea las asignaciones con flancos de reloj antes de esperar la señal de fin.

```

1 procedure run_test(val : std_logic_vector(19 downto 0)) is
2 begin
3     wait until rising_edge(clk); -- Alineacion con el reloj
4     bin    <= val;
5     start <= '1';
6     wait until rising_edge(clk); -- done baja en este flanco
7     start <= '0';
8     wait until done = '1';       -- esperar fin de conversion
9     wait for clk_period * 2;     -- margen para visualizar en waveform
10 end procedure;
```

Listing 3: Procedimiento auxiliar del testbench secuencial

Para ejecutar la simulación completa en ModelSim 5.1 basta con introducir el comando `run 8000 ns` en la consola. Los casos de prueba se resumen en la Tabla 3.

5. Conclusiones

Se han diseñado, implementado y verificado satisfactoriamente los dos conversores de código que forman la base de la Fase 1. El módulo `bcd_to_bin` resuelve la conversión mediante lógica combinacional con desplazamientos, compatible con las librerías exigidas por la asignatura. El

Binario (dec)	BCD esperado (hex)	Propósito
0	0x000000	Cero absoluto
1	0x000001	Unidad mínima
10	0x000010	Decena exacta
100	0x000100	Centena exacta
255	0x000255	Potencia de 2 menos 1
999	0x000999	Máximo operando
1 000	0x001000	Paso de millar
9 999	0x009999	Máximo de 4 dígitos
998 001	0x998001	Máximo resultado (999×999)

Cuadro 3: Casos de prueba del testbench `tb_bin_to_bcd`.

módulo `bin_to_bcd` implementa el algoritmo *Double Dabble* en una FSM de tres estados con una latencia fija de 20 ciclos de reloj, suficiente para convertir el valor máximo esperado de 998 001. Ambos módulos han superado satisfactoriamente todos los casos de prueba definidos en sus respectivos testbenches.